

1. Giới thiệu chung

Chương trình được viết bằng Assembly RISC-V, chạy trên môi trường mô phỏng có Memory-Mapped I/O, nhằm mục đích:

- Nhận lệnh Assembly RISC-V từ bàn phím
- Phân tích cú pháp lệnh (syntax checking)
- Kiểm tra:
 - Opcode có hợp lệ hay không
 - Các operand (register, immediate, label, address)
- Phân loại lệnh theo instruction type
- Thông báo lỗi cụ thể hoặc xác nhận cú pháp đúng

2. Cấu trúc dữ liệu (.data)

2.1. Bộ đệm và token

buffer: .space 256

token: .space 64

- buffer: lưu toàn bộ dòng lệnh người dùng nhập
- token: lưu từng token sau khi tách (opcode hoặc operand)

2.2. Chuỗi thông báo

msg_input: .string "\nEnter instruction: "

msg_valid_op: .string "\nOpcode: "

msg_valid_ok: .string ", valid.\n"

msg_err_op: .string "\nError: Invalid or unsupported opcode."

msg_err_op1: .string "Error: Invalid Operand 1."

msg_err_op2: .string "Error: Invalid Operand 2."

msg_err_op3: .string "Error: Invalid Operand 3."

msg_success: .string "Syntax is valid."

Các chuỗi này dùng để thông báo trạng thái phân tích cú pháp cho người dùng.

2.3. Xử lý backspace khi nhập

backspace_seq: .byte 8, 32, 8, 0

Chuỗi này lần lượt:

- 8 : backspace
- 32 : space (xóa ký tự hiển thị)
- 8 : backspace quay lại
- 0 : kết thúc

→ Giúp xóa ký tự đã nhập đúng như terminal thật.

3. Bảng opcode và phân loại lệnh

3.1. Các chuỗi opcode

```
op_lw: .string "lw"
op_lb: .string "lb"
op_lh: .string "lh"
op_lbu: .string "lbu"
op_lhu: .string "lhu"
op_sw: .string "sw"
op_sb: .string "sb"
op_sh: .string "sh"
op_lui: .string "lui"
op_auipc: .string "auipc"
op_addi: .string "addi"
op_slli: .string "slli"
op_srli: .string "srli"
op_srai: .string "srai"
op_andi: .string "andi"
op_ori: .string "ori"
op_xori: .string "xori"
op_slti: .string "slti"
op_sltiu: .string "sltiu"
```

```
op_jalr: .string "jalr"
op_add: .string "add"
op_sub: .string "sub"
op_sll: .string "sll"
op_slt: .string "slt"
op_sltu: .string "sltu"
op_xor: .string "xor"
op_srl: .string "srl"
op_sra: .string "sra"
op_or: .string "or"
op_and: .string "and"
op_beq: .string "beq"
op_bne: .string "bne"
op_blt: .string "blt"
op_bge: .string "bge"
op_bltu: .string "bltu"
op_bgeu: .string "bgeu"
op_jal: .string "jal"
```

Mỗi opcode được lưu dưới dạng chuỗi ASCII để so sánh khi parse.

3.2. Bảng ánh xạ opcode → type

```
opcode_map:
.word op_lw, 1
.word op_lb, 1
.word op_lh, 1
.word op_lbu, 1
.word op_lhu, 1
.word op_sw, 10
.word op_sb, 10
.word op_sh, 10
.word op_lui, 2
.word op_auipc, 2
.word op_addi, 3
.word op_slli, 7
.word op_srli, 7
.word op_srai, 7
.word op_andi, 3
.word op_ori, 3
.word op_xori, 3
.word op_slti, 3
.word op_sltiu, 3
.word op_jalr, 3
.word op_add, 4
.word op_sub, 4
.word op_sll, 4
```

```

.word op_slt, 4
.word op_sltu, 4
.word op_xor, 4
.word op_srl, 4
.word op_sra, 4
.word op_or, 4
.word op_and, 4
.word op_beq, 5
.word op_bne, 5
.word op_blt, 5
.word op_bge, 5
.word op_bltu, 5
.word op_bgeu, 5
.word op_jal, 6
.word 0, 0

```

- Mỗi cặp gồm:
 - Địa chỉ chuỗi opcode
 - Mã instruction type
- Dòng:

```

.word 0, 0

```

→ sentinel đánh dấu kết thúc bảng

3.3. Ý nghĩa các type lệnh

Type	Nhóm lệnh	Ví dụ
1	Load	lw rd, imm(rs1)
10	Store	sw rs2, imm(rs1)
2	U-type	lui rd, imm20
3	I-type	addi rd, rs1, imm12
4	R-type	add rd, rs1, rs2
5	Branch	beq rs1, rs2, label
6	Jump	jal rd, label

7 Shift-imm slli rd, rs1, shamt

4. Thanh ghi và I/O

4.1. Danh sách thanh ghi

regs: .string "zero", "ra", ..., "a7", ""

- Gồm đủ 32 thanh ghi RISC-V
- Chuỗi rỗng "" dùng làm điểm dừng khi duyệt

4.2. Memory-Mapped I/O

```
.eqv KEYBOARD_CTRL 0xFFFF0000
.eqv KEYBOARD_DATA 0xFFFF0004
.eqv DISPLAY_CTRL 0xFFFF0008
.eqv DISPLAY_DATA 0xFFFF000C
```

- Đây là địa chỉ phần cứng giả lập
- Bắt buộc dùng đúng giá trị do môi trường mô phỏng quy định

5. Luồng chính của chương trình (main)

5.1. In lời nhắc nhập lệnh

```
program_loop:
    la s0, msg_input
```

→ In chuỗi "Enter instruction:" ra màn hình

5.2. Nhập dữ liệu từ bàn phím

```
input_loop:
    li t0, KEYBOARD_CTRL
    lw t1, 0(t0)
    andi t1, t1, 0x1
```

- Kiểm tra bit sẵn sàng
- Đọc ký tự từ `KEYBOARD_DATA`
- Xử lý:
 - Backspace
 - Enter
 - Ký tự thường

5.3. Kết thúc nhập – bắt đầu parse

start_parse:

sb zero, 0(s0)

la s1, buffer

jal get_next_token

- Kết thúc chuỗi
- Bắt đầu phân tích token đầu tiên (opcode)

6. Tách token (get_next_token)

skip_whitespace:

beq t2, ',', skip_ws_inc

beq t2, ',', skip_ws_inc

Token được phân cách bởi:

- Space
- Tab
- Dấu phẩy
- Newline

7. Tìm opcode (find_opcode)

```
find_loop:  
    lw t1, 0(t0)  
    beqz t1, find_not_found
```

- Duyệt opcode_map
- So sánh từng ký tự của opcode
- Trả về:
 - a0 = type nếu tìm thấy
 - a0 = -1 nếu không tìm thấy

8. Kiểm tra register

8.1. Dạng xN

```
check_x_register:  
    blt t1, '0', ir_fail  
    bgt t1, '9', ir_fail  
    bgt t3, 31, ir_fail
```

Chỉ chấp nhận x0 → x31
Không phải imm5

8.2. Dạng ABI name

```
ir_scan:  
    la t1, regs
```

So khớp với bảng regs

9. Parse immediate (parse_immediate)

9.1. Hỗ trợ

- Decimal: 123, -45
- Hex: 0x1F, -0x10

9.2. Quy trình

'-' → check dấu âm

'0x' → hex

else → decimal

9.3. Công thức

- Decimal:

$value = value * 10 + digit$

- Hex:

$value = value * 16 + digit$

10. Kiểm tra giới hạn immediate

check_is_5bit_imm: $0 \rightarrow 31$

check_is_12bit_imm: $-2048 \rightarrow 2047$

check_is_20bit_imm: $-524288 \rightarrow 524287$

11. Kiểm tra label

check_is_label:

chữ / '_' đầu

sau đó chữ, số hoặc '_'

Ngoài ra:

jal find_opcode

→ Không cho label trùng opcode

12. Kiểm tra address (offset(register))

check_is_address:
offset(rs)

- Offset: imm12
- Register: hợp lệ
- Không dư ký tự

13. Kiểm tra operand theo type

Ví dụ R-type:

type_r:
jal check_reg_op1
jal check_reg_op2
jal check_reg_op3

Mỗi type có chuỗi kiểm tra riêng, đúng thứ tự operand.

14. Thông báo lỗi & kết thúc

e_op1 → Error Operand 1
e_op2 → Error Operand 2
e_op3 → Error Operand 3

Nếu thành công:

success:
"Syntax is valid."

15. Cơ chế hỏi người dùng có tiếp tục chương trình hay không

Sau khi chương trình hoàn thành việc kiểm tra cú pháp lệnh (thành công hoặc báo lỗi), chương trình không kết thúc ngay mà hỏi người dùng có muốn tiếp tục nhập lệnh mới hay không.

15.1. In thông báo hỏi tiếp tục

done:

```
la a0, msg_continue
jal print_string
```

Chuỗi được in ra là:

```
msg_continue: .string "\n\nContinue? (1=Yes, 0=No): "
```

➡ Người dùng được yêu cầu nhập:

- 1 → tiếp tục chương trình
- 0 → kết thúc chương trình

16.2. Chờ người dùng nhập lựa chọn

wait_continue_input:

```
li t0, KEYBOARD_CTRL
lw t1, 0(t0)
andi t1, t1, 0x1
beqz t1, wait_continue_input
```

- Chương trình polling bàn phím
- Chỉ tiếp tục khi có phím được nhấn

16.3. Phân nhánh theo giá trị nhập

```
li t3, '1'
beq t2, t3, got_one
li t3, '0'
beq t2, t3, got_zero
j wait_continue_input
```

Phím

Hành vi

'1' Tiếp tục

'0' Thoát

Khác Bỏ qua, chờ nhập lại

16.4. Trường hợp tiếp tục (got_one)

got_one:

sw '1' → DISPLAY

chờ Enter

in newline

j program_loop

Trích dẫn:

j program_loop

➡ Chương trình quay lại từ đầu, yêu cầu nhập lệnh mới.

16.5. Trường hợp kết thúc (got_zero)

got_zero:

sw '0' → DISPLAY

chờ Enter

in newline

j exit_program

16.6. Thoát chương trình

exit_program:

la a0, msg_exit

jal print_string

li a7, 10

ecall

- In thông báo:

msg_exit: .string "\nProgram ended\n"

- Gọi syscall `ecall` để kết thúc chương trình